AIM2 Preparation

Week 1 - Day 6: Files, Exceptions & Modules

5-Minute Review Notes | File I/O, try/except & Reusable Python Modules

Day 6 Goal

Learn how to save and read data from files, safely handle runtime errors, and organize reusable functions into separate Python modules.

1. File Handling

Files allow data to persist after a Python program finishes. The basic workflow is: **open -> read/write -> close**. Using with automatically closes the file.

2. Writing to a Text File

```
with open("students.txt", "w") as file:
    file.write("Krishna,85,92,88\n")
    file.write("Rahul,75,80,70\n")
```

Mode **w** opens a file for writing. If the file already exists, its existing content is replaced.

3. Reading an Entire File

```
with open("students.txt", "r") as file:
    content = file.read()

print(content)
```

Mode **r** opens a file for reading. read() loads the file content as text.

4. Reading a File Line by Line

```
with open("students.txt", "r") as file:
    for line in file:
        print(line.strip())
```

Looping through the file processes one line at a time. strip() removes leading/trailing whitespace, including the newline at the end of each line.

5. Common File Modes

Mode	Purpose
r	Read an existing file
w	Write; creates or overwrites a file
a	Append new content to the end
x	Create a new file; fails if it already exists

6. What Is an Exception?

An exception is an error that occurs while a program is running. Instead of letting the program immediately crash, Python can catch expected errors and respond safely.

7. try / except

```
try:
    age = int(input("Enter your age: "))
except ValueError:
    print("Please enter a valid number.")
```

Python first runs the try block. If int() receives invalid text such as "abc", it raises ValueError and the matching except block runs.

8. Why Catch Specific Exceptions?

```
except ValueError:
```

Catching a specific exception makes the program clearer and prevents unrelated programming errors from being hidden accidentally.

9. Modules

A module is a Python file containing reusable code such as functions. It helps split a larger program into smaller files with clear responsibilities.

student_utils.py

```
def calculate_total(math, python, ai):
    return math + python + ai

def calculate_average(total):
    return total / 3

def calculate_grade(average):
    if average >= 90:
        return "A"
    elif average >= 80:
        return "B"
    else:
        return "C or below"
```

10. Importing Your Module

```
import student_utils

total = student_utils.calculate_total(85, 92, 88)
```

The module name identifies the file, and the dot accesses a function inside that module.

Importing Specific Functions

```
from student_utils import calculate_total, calculate_average

total = calculate_total(85, 92, 88)
```

11. Combining Files + Functions + Modules

A realistic program can read student records from a text file, clean each line, convert scores to numbers, pass the scores to reusable functions, and print the calculated result.

```
for line in file:
    line = line.strip()
    # split / convert values
    # call reusable functions
    # display result
```

12. Indentation Reminder

```
with open("students.txt", "r") as file:
    for line in file:
        print(line.strip())
```

Indentation shows which statements belong to with, for, try, except, functions, and conditions. It is part of Python syntax, not just formatting.

Day 6 Practice Highlights

- Created and wrote data to students.txt.
- Read text files and processed them line by line.
- Used strip() while reading lines.
- Handled invalid age input using try/except ValueError.
- Created a custom student_utils module.
- Added calculate_total(), calculate_average(), and calculate_grade().
- Imported and reused functions from another Python file.
- Reinforced the importance of indentation.

Quick Syntax Cheat Sheet

```
with open("file.txt", "w") as file:
    file.write("text\n")

with open("file.txt", "r") as file:
    for line in file:
        print(line.strip())

try:
    value = int(input("Value: "))
except ValueError:
    print("Invalid input")

import student_utils
from student_utils import calculate_total
```

Key Takeaways

- Files let programs store and retrieve persistent data.
- with open(...) is preferred because the file is closed automatically.
- strip() is useful when processing text lines.
- try/except lets a program handle expected runtime errors safely.
- Catch specific exceptions such as ValueError when possible.
- Modules organize reusable code into separate Python files.
- Functions + modules reduce duplication and improve maintainability.
- Files, exceptions, and modules are important building blocks for larger data-processing applications.

Progress

Week 1 Day 6 - Files, Exceptions & Modules: COMPLETE
Practice focus: students.txt, ValueError handling, and the student_utils custom module.